



南開大學
Nankai University

计算机学院
并行程序设计实验报告

NTT 的 Pthread 与 OpenMP 优化

姓名：强徽东

学号：2411764

专业：计算机科学与技术

2026 年 5 月 29 日

目录

1 实验环境	2
2 代码结构	2
3 实验一：基础 NTT 的 Pthread 优化	2
3.1 NTT 原理	2
3.2 蝶形运算并行	3
4 实验二：CRT 实现大模数 NTT	6
4.1 为什么需要 CRT	6
4.2 CRT 原理	7
4.3 CRT 合并原理	8
4.4 模数选择	11
5 实验三：CRT 的 Pthread 优化	11
5.1 并行思路	11
5.2 CRT 合并并行	12
6 实验四：OpenMP 自动调度优化	12
6.1 OpenMP 运行时设置	12
6.2 内部 NTT 并行	13
6.3 模数并行	15
6.4 CRT 合并中的 guided 调度	16
7 实验五：OpenMP SIMD 自动向量化	16
7.1 SIMD 优化动机	16
7.2 消除旋转因子递推依赖	17
7.3 小模数乘法优化	18
8 性能测试与正确性分析	18
8.1 性能测试与结果分析	18
8.2 Pthread 和 OpenMP 区别分析	20
9 其他算法优化策略	20
9.1 任务池与动态负载均衡	20
9.2 分层混合并行	21
9.3 缓存友好的分块优化	21
9.4 多次实验任务并行	21
10 问题与反思	21
10.1 关于三模数与四模数	21
10.2 关于模数并行与内部并行	21
11 总结	22

1 实验环境

本实验选题为 NTT (Number Theoretic Transform, 数论变换) 方向, 主要围绕多项式乘法在不同模数下的并行优化展开。实验程序在 OpenEuler 服务器平台提交运行, 每个任务申请 1 个 CPU, 该 CPU 具有 8 个核心。根据实验指导书要求, 线程数量并不是越多越好, 最佳方式是最多创建 7 个子线程, 并让主线程也承担一部分计算任务, 从而形成最多 8 个工作线程。

表 1: 实验环境与约束

类别	配置或说明
实验平台	OpenEuler 服务器平台
操作系统	Linux / aarch64 环境
编程语言	C++
主要并行工具	Pthread、OpenMP

2 代码结构

实验过程中按优化顺序实现了多个版本, 保留为独立文件, 便于分别测试和比较。

表 2: 主要代码文件

文件名	说明
main_ntt_pthread.cc	基础 NTT 的 Pthread 优化版本, 对蝶形循环进行并行。
main_crt4_serial.cc	四模数 CRT 串行版本, 用于覆盖大模数。
main_crt4_pthread.cc	四模数 CRT 的 Pthread 版本, 3 个子线程加主线程处理 4 个模数。
main_crt4_openmp.cc	OpenMP 内部 NTT 并行版本。
main_crt4_openmp_modparallel.cc	OpenMP 模数并行版本。
main_crt4_openmp_modparallel_simd.cc	模数并行基础上加入 OpenMP SIMD 自动向量化提示。

3 实验一：基础 NTT 的 Pthread 优化

3.1 NTT 原理

NTT 是 FFT 在有限域上的对应形式。其核心思想是将多项式从系数表示转化为点值表示, 使卷积变为逐点乘法:

$$\begin{aligned}
 A(x), B(x) &\xrightarrow{\text{NTT}} \hat{A}, \hat{B} \\
 \hat{C}[i] &= \hat{A}[i] \hat{B}[i] \pmod{P} \\
 C(x) &\xleftarrow{\text{INTT}} \hat{C}
 \end{aligned}$$

对于长度扩展到 `limit` 的数组, NTT 的复杂度为:

$$O(\text{limit} \log \text{limit})$$

3.2 蝶形运算并行

基础 NTT 的循环结构如下：

NTT 蝶形循环结构

```

1 for (int len = 2; len <= n; len <= 1) {
2     int half = len >> 1;
3     for (int base = 0; base < n; base += len) {
4         for (int k = 0; k < half; ++k) {
5             // butterfly
6         }
7     }
8 }

```

同一层内，不同 block 之间没有数据依赖，可以并行；但第 $s+1$ 层依赖第 s 层结果，因此层与层之间必须同步。本实验的 Pthread 版本使用 `pthread_barrier_t` 在每一层结束后同步。

更具体地说，在长度为 `len` 的某一层蝶形变换中，一个 block 覆盖区间：

$$[base, base + len)$$

其中前半部分和后半部分两两配对进行蝶形计算：

$$x = f[base + k]$$

$$y = f[base + k + half] \cdot w_k \pmod{p}$$

$$f[base + k] = x + y \pmod{p}$$

$$f[base + k + half] = x - y \pmod{p}$$

对于同一层来说，不同 block 对应的数组区间完全不重叠。例如 block 0 修改的是 $[0, len)$ ，block 1 修改的是 $[len, 2len)$ ，二者不会写同一位置。因此同一层内部可以安全并行，不需要互斥锁。但下一层的 `len` 会翻倍，一个更大的 block 会使用上一层多个小 block 的结果，所以必须保证所有线程完成当前层后，才能进入下一层。

本实验中使用一个上下文结构体保存 NTT 线程共享的信息，包括数组地址、模数、当前线程数以及 barrier：

NTT 线程共享上下文

```

1 struct NttContext {
2     u64 *f;
3     int n;
4     int threads;
5     u64 mod;
6     bool inverse;
7     pthread_barrier_t barrier;
8 };
9
10 struct NttTask {

```

```

11     NttContext *ctx;
12     int tid;
13 };

```

每个线程通过自己的 tid 决定负责哪些 block。前中期 block 数量较多时，采用轮转分配：

按 block 轮转分配蝶形块

```

1 void ntt_stage_by_blocks(u64 *f, int n, int len,
2                          u64 wn, u64 mod, int tid, int threads) {
3     int half = len >> 1;
4     int blocks = n / len;
5
6     for (int block = tid; block < blocks; block += threads) {
7         int base = block * len;
8         u64 w = 1;
9         for (int k = 0; k < half; ++k) {
10             u64 x = f[base + k];
11             u64 y = mul_mod(f[base + k + half], w, mod);
12             f[base + k] = add_mod(x, y, mod);
13             f[base + k + half] = sub_mod(x, y, mod);
14             w = mul_mod(w, wn, mod);
15         }
16     }
17 }

```

这里使用 `block += threads` 而不是简单地给每个线程一大段连续 block，是为了让不同线程获得更均匀的任务数量。例如 8 个线程时，第 0 个线程处理第 0、8、16 个 block，第 1 个线程处理第 1、9、17 个 block。由于同一层每个 block 的蝶形数量都等于 `half`，这种划分可以较好地保证负载均衡。

当 NTT 进入后几层时，`blocks=n/len` 会变得很小，甚至只有 1 个。如果仍然按 block 分配，就会出现只有一个线程工作、其他线程全部等待的情况。因此实现中又增加了按 butterfly 总数划分的策略。每一层总共有 $n/2$ 个蝶形操作，将这些蝶形操作按线程数均分：

后期按 butterfly 数量划分

```

1 void ntt_stage_by_butterflies(u64 *f, int n, int len,
2                               u64 wn, u64 mod, int tid, int threads) {
3     int half = len >> 1;
4     int total = n >> 1;
5     int l = total * tid / threads;
6     int r = total * (tid + 1) / threads;
7
8     while (l < r) {
9         int block = l / half;
10        int k = l % half;
11        int block_end = std::min(r, (block + 1) * half);
12        int base = block * len;
13        u64 w = qpow(wn, k, mod);
14
15        for (; l < block_end; ++l, ++k) {

```

```

16         u64 x = f[base + k];
17         u64 y = mul_mod(f[base + k + half], w, mod);
18         f[base + k] = add_mod(x, y, mod);
19         f[base + k + half] = sub_mod(x, y, mod);
20         w = mul_mod(w, wn, mod);
21     }
22 }
23 }

```

该策略的关键是把逻辑上的二维循环 `block-k` 展开为一维编号 `idx`。某个 butterfly 编号 `idx` 对应：

$$block = \left\lfloor \frac{idx}{half} \right\rfloor, \quad k = idx \bmod half$$

这样即使某一层只有 1 个 block，也能把该 block 内部的大量 butterfly 分给多个线程，从而避免后期负载严重不均。

完整的线程执行函数如下。所有线程在同一层内执行各自负责的 butterfly，执行完之后调用 `pthread_barrier_wait` 等待其他线程。只有所有线程都到达 barrier 后，才会一起进入下一层。

NTT 线程函数中的层间同步

```

1 void run_ntt_worker(NttContext *ctx, int tid) {
2     u64 *f = ctx->f;
3     int n = ctx->n;
4     int threads = ctx->threads;
5     u64 mod = ctx->mod;
6
7     for (int len = 2; len <= n; len <= 1) {
8         u64 wn = qpow(G, (mod - 1) / len, mod);
9         if (ctx->inverse) wn = qpow(wn, mod - 2, mod);
10
11         int blocks = n / len;
12         if (blocks >= threads) {
13             ntt_stage_by_blocks(f, n, len, wn, mod, tid, threads);
14         } else {
15             ntt_stage_by_butterflies(f, n, len, wn, mod, tid, threads);
16         }
17
18         pthread_barrier_wait(&ctx->barrier);
19     }
20
21     if (ctx->inverse) {
22         u64 inv_n = qpow((u64)n, mod - 2, mod);
23         for (int i = tid; i < n; i += threads) {
24             f[i] = mul_mod(f[i], inv_n, mod);
25         }
26         pthread_barrier_wait(&ctx->barrier);
27     }
28 }

```

这里的同步是必要的。若没有 barrier，某个较快线程可能已经进入下一层，并读取其他线程尚未完成更新的数据，从而破坏 NTT 的层间依赖关系，导致结果错误。由于每一层内部的写入区间互不重叠，同一层内部不需要锁；而层与层之间只需要一次 barrier 同步。因此该实现的同步开销主要是每一层一次 barrier，整体同步次数约为：

$$\log_2 n$$

对于 $n = 131072$ ，NTT 层数约为 18 层，同步次数可以接受。

线程创建时同样遵循实验要求：总工作线程数不超过 8，主线程也参与计算。只创建 `threads-1` 个子线程，主线程使用最后一个 tid 直接调用 `run_ntt_worker`：

创建子线程并让主线程参与 NTT

```

1 int threads = get_thread_count(n >> 1);
2 int worker_threads = threads - 1;
3
4 pthread_barrier_init(&ctx.barrier, nullptr, threads);
5
6 for (int t = 0; t < worker_threads; ++t) {
7     tasks[t] = {&ctx, t};
8     pthread_create(&tids[t], nullptr, ntt_worker, &tasks[t]);
9 }
10
11 run_ntt_worker(&ctx, worker_threads);
12
13 for (int t = 0; t < worker_threads; ++t) {
14     pthread_join(tids[t], nullptr);
15 }
16
17 pthread_barrier_destroy(&ctx.barrier);

```

综上，基础 NTT 的 Pthread 并行主要利用了两个事实：第一，同一层的蝶形操作之间没有跨 block 写冲突，可以并行；第二，不同层之间存在严格的数据依赖，必须使用 barrier 做层间同步。通过 block 分配与 butterfly 分配两种策略结合，可以在 NTT 前中后期都保持较好的线程利用率。由于同层每个蝶形变换计算量基本相同，所以这里没有使用任务池的方式进行优化。

4 实验二：CRT 实现大模数 NTT

4.1 为什么需要 CRT

实验数据中出现了大模数，例如：

263882790666241

以及测试中出现的：

1337006139375617

这些模数虽然可以用 `uint64_t` 存储，但不太适合直接做 NTT。

因此我们选取若干个 NTT 友好的小模数，分别在小模数下做 NTT，最后通过中国剩余定理合并。

4.2 CRT 原理

CRT (Chinese Remainder Theorem, 中国剩余定理) 的作用是：当我们无法或不方便直接在目标模数 P 下进行 NTT 时，先把问题转移到若干个较小且适合 NTT 的模数下分别求解，再把这些小模数下的结果合并回目标模数下。

在本实验中，目标是计算：

$$C(x) = A(x)B(x) \pmod{P}$$

如果 P 很大，或者 P 不是形如 $c \cdot 2^k + 1$ 的 NTT 友好模数，则很难直接找到足够高阶的单位根来完成 NTT。于是选取若干个互不相同且两两互质的 NTT 友好小模数：

$$p_1, p_2, \dots, p_m$$

这些小模数满足：

$$p_i = c_i \cdot 2^{k_i} + 1$$

因此可以在每个 p_i 下分别进行快速 NTT。

设真实的整数卷积结果为：

$$D(x) = A(x)B(x)$$

其中每个系数 $D[k]$ 是没有取目标模数之前的整数卷积系数。对于每个小模数 p_i ，我们通过 NTT 计算：

$$C_i(x) = D(x) \pmod{p_i}$$

也就是说：

$$C_i[k] \equiv D[k] \pmod{p_i}$$

因此，对于每个系数 $D[k]$ ，我们可以得到它在多个小模数下的余数：

$$D[k] \bmod p_1, \quad D[k] \bmod p_2, \quad \dots, \quad D[k] \bmod p_m$$

中国剩余定理保证：如果 $p_1, p_2, \dots, p_m$ 两两互质，那么在模

$$M = p_1 p_2 \cdots p_m$$

意义下，存在唯一的整数 x 满足：

$$x \equiv C_i[k] \pmod{p_i}, \quad i = 1, 2, \dots, m$$

如果 M 足够大，使得真实整数卷积系数满足：

$$0 \leq D[k] < M$$

那么 CRT 恢复出的 x 就不是单纯的模 M 余数，而是原始整数卷积系数 $D[k]$ 本身。最后再计算：

$$ans[k] = D[k] \bmod P$$

即可得到目标模数 P 下的多项式乘法结果。

整体流程可以概括为：

1. 选取若干个 NTT 友好小模数 p_i ；
2. 对每个小模数 p_i ，将输入多项式系数映射到模 p_i 下；
3. 分别执行 NTT、点值乘法和逆 NTT，得到 $C_i(x)$ ；
4. 对每个结果系数位置 k ，使用 CRT 合并 $C_1[k], C_2[k], \dots, C_m[k]$ ；
5. 将恢复出的整数系数对目标模数 P 取模。

对应的代码结构如下：

CRT NTT 的整体流程

```

1 void poly_multiply_crt(u64 *a, u64 *b, u64 *ab,
2                         int n, u64 target_mod) {
3     int result_len = 2 * n - 1;
4     vector<vector<u64>> residues(CRT_CNT);
5
6     for (int i = 0; i < CRT_CNT; ++i) {
7         multiply_mod_ntt(a, b, residues[i], n, CRT_MODS[i]);
8     }
9
10    for (int i = 0; i < result_len; ++i) {
11        // combine residues[0][i], residues[1][i], ...
12        ab[i] = crt_combine_one(..., target_mod);
13    }
14 }

```

可以看到，CRT 并没有改变 NTT 本身的数学过程，而是把一次大模数卷积分解为多次小模数卷积。这样做的好处是每个小模数都可以选择成熟的 NTT 友好质数，避免直接处理大模数单位根的问题。

4.3 CRT 合并原理

设选取 m 个两两互质的小模数：

$$p_1, p_2, \dots, p_m$$

分别计算：

$$C_i(x) = A(x)B(x) \pmod{p_i}$$

对于某个结果系数 $c[k]$ ，已知：

$$c[k] \equiv C_1[k] \pmod{p_1}$$

$$c[k] \equiv C_2[k] \pmod{p_2}$$

...

$$c[k] \equiv C_m[k] \pmod{p_m}$$

CRT 可以在模：

$$M = \prod_{i=1}^m p_i$$

意义下唯一恢复 $c[k]$ 。恢复后再对目标模数 P 取模，得到最终答案。

本实验采用的是逐步合并的写法。假设已经合并了前 $i-1$ 个模数，当前得到的数为 x ，已经满足：

$$x \equiv C_j[k] \pmod{p_j}, \quad j = 1, 2, \dots, i-1$$

同时令：

$$prod = p_1 p_2 \cdots p_{i-1}$$

接下来希望加入第 i 个模数 p_i 。新的数可以写成：

$$x' = x + prod \cdot t$$

因为 $prod$ 是前面所有模数的乘积，所以 x' 对前 $i-1$ 个模数的余数不会改变。现在只需要选择 t ，使得：

$$x + prod \cdot t \equiv C_i[k] \pmod{p_i}$$

移项可得：

$$prod \cdot t \equiv C_i[k] - x \pmod{p_i}$$

由于 p_i 与 $prod$ 互质， $prod$ 在模 p_i 意义下存在逆元：

$$prod^{-1} \pmod{p_i}$$

因此：

$$t \equiv (C_i[k] - x) \cdot prod^{-1} \pmod{p_i}$$

代码中对应为：

CRT 单步合并公式

```
1 u64 cur = (u64)(x % mi);
2 u64 need = (r[i] + mi - cur) % mi;
3 u64 t = mul_mod(need, inv_prod_mod[i], mi);
4 x += prod * t;
5 prod *= mi;
```

其中 $r[i]$ 表示当前系数在第 i 个小模数下的余数，即 $C_i[k]$ ； cur 是当前已经合并出的 x 在模 p_i 下的余数； $need$ 表示还需要补上的差值； $inv_prod_mod[i]$ 是 $prod$ 关于 p_i 的逆元。

为了避免在每个系数合并时重复计算逆元，程序在合并前预处理所有逆元：

预处理 CRT 合并所需逆元

```
1 void prepare_crt_inverse(u64 inv_prod_mod[CRT_CNT]) {
2     u128 prod = CRT_MODS[0].mod;
3     inv_prod_mod[0] = 1;
4     for (int i = 1; i < CRT_CNT; ++i) {
```

```

5     u64 mi = CRT_MODS[i].mod;
6     inv_prod_mod[i] =
7         qpow((u64)(prod % mi), mi - 2, mi);
8     prod *= mi;
9 }
10 }

```

这里使用快速幂：

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

是因为选取的小模数均为质数，可以由费马小定理求逆元。

单个系数的完整 CRT 合并代码如下：

单个系数的 CRT 合并

```

1 u64 crt_combine_one(const u64 r[CRT_CNT],
2                     u64 target_mod,
3                     const u64 inv_prod_mod[CRT_CNT]) {
4     u128 x = r[0];
5     u128 prod = CRT_MODS[0].mod;
6
7     for (int i = 1; i < CRT_CNT; ++i) {
8         u64 mi = CRT_MODS[i].mod;
9         u64 cur = (u64)(x % mi);
10        u64 need = (r[i] + mi - cur) % mi;
11        u64 t = mul_mod(need, inv_prod_mod[i], mi);
12        x += prod * t;
13        prod *= mi;
14    }
15
16    return (u64)(x % target_mod);
17 }

```

这里 `x` 和 `prod` 使用 `__uint128_t` 类型，是因为四个小模数乘积约为 5.77×10^{35} ，远远超过 `uint64_t` 能表示的范围。虽然最终答案会对目标模数取模后存回 `uint64_t`，但合并过程中的中间值必须使用 128 位整数保存。

CRT 合并是按系数独立进行的。也就是说，`ab[0]` 的合并只依赖各小模数结果的第 0 项，`ab[1]` 的合并只依赖第 1 项，以此类推。因此无论是在 Pthread 版本还是 OpenMP 版本中，都可以将结果区间划分给不同线程并行合并：

按输出系数并行进行 CRT 合并

```

1 for (int i = 1; i < r; ++i) {
2     u64 r[CRT_CNT];
3     for (int j = 0; j < CRT_CNT; ++j) {
4         r[j] = residues[j][i];
5     }
6     ab[i] = crt_combine_one(r, target_mod, inv_prod_mod);
7 }

```

由于每个线程写入的 `ab[i]` 区间不同，CRT 合并阶段不需要互斥锁。相比 NTT 蝶形运算需要层间同步，CRT 合并阶段只有任务开始前和结束后的同步，结构更简单，也更适合直接做数据并行。

4.4 模数选择

最终选择使用 4 个模数，模数选取为：

表 3: 模数 CRT 中使用的 NTT 友好模数

编号	模数	原根
1	998244353	3
2	1004535809	3
3	469762049	3
4	1224736769	3

四个模数乘积约为：

$$5.77 \times 10^{35}$$

能够覆盖 $n = 131072$ 、 $P = 1337006139375617$ 下的最坏卷积系数上界。

实验中曾尝试使用 754974721 作为第四模数，但最后一个大模数样例错误。替换为更大的 1224736769 后，结果正确。这说明 CRT 中模数乘积的余量会直接影响大模数样例的可靠性。

5 实验三：CRT 的 Pthread 优化

5.1 并行思路

CRT 版本天然具有粗粒度并行性。不同小模数下的 NTT 卷积互不依赖：

$$C_i = A \cdot B \pmod{p_i}$$

因此可以让不同线程处理不同模数。四模数版本中，使用 3 个子线程处理前三个小模数，主线程处理第四个小模数：

四模数任务并行

```

1  const int worker_threads = CRT_CNT - 1;
2
3  for (int i = 0; i < worker_threads; ++i) {
4      pthread_create(&tids[i], nullptr, mod_worker, &tasks[i]);
5  }
6
7  run_mod_worker(&tasks[worker_threads]);
8
9  for (int i = 0; i < worker_threads; ++i) {
10     pthread_join(tids[i], nullptr);
11 }

```

5.2 CRT 合并并行

小模数 NTT 结果全部得到后，每个输出系数的 CRT 合并相互独立，因此可以再次按输出区间并行。

CRT 合并任务划分

```

1 for (int t = 0; t < worker_threads; ++t) {
2     tasks[t].l = result_len * t / threads;
3     tasks[t].r = result_len * (t + 1) / threads;
4     pthread_create(&tids[t], nullptr, combine_worker, &tasks[t]);
5 }
6
7 tasks[worker_threads].l = result_len * worker_threads / threads;
8 tasks[worker_threads].r = result_len;
9 run_combine_worker(&tasks[worker_threads]);

```

由于每个线程写不同的 `ab[i]`，因此不需要互斥锁。

6 实验四：OpenMP 自动调度优化

6.1 OpenMP 运行时设置

OpenMP 版本中使用如下初始化：

OpenMP 初始化

```

1 void init_openmp_runtime() {
2     omp_set_dynamic(1);
3     omp_set_num_threads(get_omp_threads());
4     omp_set_schedule(omp_sched_guided, 64);
5 }

```

`omp_set_dynamic(1)` 允许 OpenMP 根据任务情况调整实际线程数；`omp_set_num_threads` 设置最多使用 8 个线程；`omp_set_schedule` 设置 `schedule(runtime)` 时使用 `guided` 调度。

与 Pthread 版本相比，OpenMP 的主要优势是将线程创建、任务划分和线程回收交给编译器与运行时系统管理。Pthread 版本需要手动维护 `pthread_t`、任务结构体、区间划分以及 `pthread_join`；而 OpenMP 只需要在关键循环前添加并行制导语句，运行时会自动创建线程组并完成循环迭代分配。因此，本实验中的 OpenMP 版本主要用于探索“自动调度”对 NTT 和 CRT 这类循环密集型程序的优化效果。

这里仍然限制最大线程数为 8。若线程数超过核心数，额外线程会与已有线程竞争核心资源，可能引入上下文切换和缓存扰动。与前面的 Pthread 实现一致，OpenMP 版本也遵循“最多使用 8 个工作线程”的原则。不同的是，Pthread 版本采用“最多 7 个子线程加主线程参与”的形式，而 OpenMP 中主线程会自动成为并行区域中的一个工作线程，因此只需要设置总线程数即可。

线程数的获取部分也进行了修改。程序默认使用 8 个线程，同时允许通过环境变量 `OMP_NUM_THREADS` 调整线程数，最后再用 `std::min` 截断到 8，避免超过实验平台限制：

OpenMP 线程数限制

```

1 int get_omp_threads() {
2     int threads = 8;
3     const char *env = std::getenv("OMP_NUM_THREADS");
4     if (env != nullptr) {
5         int v = std::atoi(env);
6         if (v > 0) threads = v;
7     }
8     return std::min(threads, 8);
9 }

```

6.2 内部 NTT 并行

第一种 OpenMP 策略是四个模数串行处理,但每个小模数内部的 NTT 蝶形循环使用 OpenMP 并行。对于 block 数量充足的层,使用静态调度:

NTT block 静态调度

```

1 #pragma omp parallel for schedule(static)
2 for (int block = 0; block < blocks; ++block) {
3     // butterfly block
4 }

```

`schedule(static)` 在循环开始时一次性分配任务,开销小,适合每个 block 工作量基本相同的 NTT 蝶形计算。

在 NTT 的迭代过程中,每一层由若干个长度相同的 butterfly block 组成。对于固定的一层,所有 block 的 `len`、`half` 和旋转因子计算方式完全一致,因此每个 block 的计算量基本相同。这种循环属于比较规则的循环,使用 `schedule(static)` 可以让 OpenMP 在进入循环时直接把 block 平均划分给各个线程,不需要运行时反复维护任务队列,调度开销较低。

实际代码中并不是所有层都直接按 block 并行,而是根据当前层的 block 数量选择不同的划分方式:

根据 block 数量选择并行粒度

```

1 if (blocks >= get_omp_threads()) {
2     #pragma omp parallel for schedule(static)
3     for (int block = 0; block < blocks; ++block) {
4         int base = block * len;
5         u64 w = 1;
6         for (int k = 0; k < half; ++k) {
7             // butterfly in one block
8         }
9     }
10 } else {
11     int total = n >> 1;
12     #pragma omp parallel
13     {
14         int tid = omp_get_thread_num();
15         int nth = omp_get_num_threads();

```

```

16     int l = total * tid / nth;
17     int r = total * (tid + 1) / nth;
18     // split all butterflies in this stage
19 }
20 }

```

这个修改主要是为了解决 NTT 后几层 block 数量不足的问题。前几层 blocks 很多，直接把 block 分给线程即可；但随着层数增加，len 变大，blocks=n/len 会变小。若某一层只有 1 个或 2 个 block，而仍然按 block 划分，很多线程会没有任务。因此在 blocks < get_omp_threads() 时，代码改为按照该层总蝶形数 total=n/2 划分区，使每个线程都能获得一段连续的 butterfly 任务。

在第二个分支中，l 和 r 的计算方式与前面 Pthread 版本中手动划分任务区间的思想一致：

$$l = \frac{\text{total} \times \text{tid}}{\text{nth}}, \quad r = \frac{\text{total} \times (\text{tid} + 1)}{\text{nth}}$$

这样可以保证即使 total 不能被线程数整除，最后几个任务也会自然分配到相邻线程中，不会遗漏，也不会重复。OpenMP 在这里负责提供线程编号 omp_get_thread_num() 和线程总数 omp_get_num_threads()，不再需要手动创建 pthread_t 数组。

内部并行的核心思想是保持 NTT 的层序依赖不变，只把同一层内部互相独立的蝶形块并行执行。第 s 层的结果会作为第 s+1 层的输入，因此不同层之间不能并行；但是在同一层内，不同 block 读写的数组区间互不重叠，一个 block 不会修改另一个 block 的数据。OpenMP 的 parallel for 在循环结束处默认带有隐式同步，能够保证所有线程完成当前层之后，程序才进入下一层。这一点相当于 Pthread 版本中的 pthread_barrier_wait，只是同步由 OpenMP 运行时自动完成。

此外，在点值乘法和逆 NTT 的归一化阶段，也加入了 OpenMP 循环并行：

点值乘法与逆变换缩放并行

```

1 #pragma omp parallel for schedule(static)
2 for (int i = 0; i < limit; ++i) {
3     A[i] = mul_mod(A[i], B[i], nm.mod);
4 }
5
6 #pragma omp parallel for schedule(static)
7 for (int i = 0; i < n; ++i) {
8     f[i] = mul_mod(f[i], inv_n, mod);
9 }

```

这两处循环的每次迭代只读写对应下标的数据，天然不存在写冲突，因此不需要临界区或互斥锁。相比只优化蝶形运算，把这两处也并行化可以让完整的小模数 NTT 乘法流程保持较高并行度。

从核心利用率看，内部 NTT 并行比单纯模数并行更有潜力。四模数 CRT 只有 4 个小模数任务，如果只在模数层并行，最多同时使用 4 个线程；而对于 n = 131072 这样的规模，单个 NTT 内部每一层都有大量蝶形运算，内部并行可以把一个模数下的计算继续拆分给更多线程，更容易利用 8 核平台。其代价是每一层都会产生一次并行循环和隐式同步，因此在很小规模或很靠后的层中，线程同步开销可能变得明显。

本实验中没有采用 OpenMP 的嵌套并行，即没有同时在“四个模数之间”和“每个模数内部”都开并行区域。这样做是为了避免线程数量成倍增加。例如外层 4 个模数并行，内层每个 NTT 又开 8 个线程，理论上会产生远多于 8 个的工作线程，不符合实验平台一个任务 8 核的限制，也容易造成资

源浪费。因此内部 NTT 并行版本选择“四个模数顺序处理、单个模数内部充分并行”的方式。

6.3 模数并行

第二种策略是模数并行，即四个小模数任务并行执行，每个小模数内部 NTT 保持串行，避免嵌套并行：

OpenMP 模数并行

```
1 int mod_threads = std::min(get_omp_threads(), CRT_CNT);
2
3 #pragma omp parallel for schedule(static) num_threads(mod_threads)
4 for (int i = 0; i < CRT_CNT; ++i) {
5     multiply_mod_ntt(a, b, residues[i], n, CRT_MODS[i]);
6 }
```

该方法最多使用 4 个线程，粒度较粗，线程开销较低。但由于平台有 8 核，它无法完全利用全部核心。与内部 NTT 并行相比，它作为并行粒度对照实验存在。

这里的关键修改是 `num_threads(mod_threads)`。由于四模数 CRT 只有 4 个模数任务，若直接使用默认 8 个线程，则会有一部分线程启动后没有实际迭代可执行，造成不必要的线程调度开销。因此代码先计算：

$$mod_threads = \min(get_omp_threads(), CRT_CNT)$$

当 `CRT_CNT=4` 时，模数并行阶段最多只启动 4 个线程。每个线程负责一个小模数下的完整 NTT 乘法，包括正变换、点值乘法和逆变换。

模数并行版本的运行时初始化也略有区别：

模数并行版本的 OpenMP 初始化

```
1 void init_omp_runtime() {
2     omp_set_dynamic(0);
3     omp_set_num_threads(get_omp_threads());
4     omp_set_schedule(omp_sched_guided, 256);
5 }
```

这里将 `omp_set_dynamic` 设为 0，是因为模数并行阶段已经通过 `num_threads(mod_threads)` 显式指定线程数。如果再允许运行时动态改变线程数，可能导致四个模数任务没有按预期并行展开。也就是说，内部 NTT 并行版本更依赖 OpenMP 根据循环规模自动调节线程，而模数并行版本更强调固定的“一模数一任务”结构。

模数并行利用的是 CRT 算法本身的独立性。对于四个小模数 p_0, p_1, p_2, p_3 ，每个模数下的 NTT 计算只依赖原始输入多项式 a, b 和当前模数，不依赖其他模数的中间结果。因此可以把四个小模数任务看成四个互相独立的大任务，分别计算：

$$C_i(x) \equiv A(x)B(x) \pmod{p_i}$$

待四个结果全部完成后，再进入 CRT 合并阶段。

这种划分的优点是同步关系非常简单。四个模数任务之间没有共享写入，每个线程只写自己的 `residues[i]` 数组，因此不需要锁，也不需要 NTT 的每一层进行跨线程同步；OpenMP 只需要

在四个模数任务全部结束后进行一次隐式同步。相比内部 NTT 并行，它的调度次数更少，代码也更接近串行 CRT 版本。

缺点也很明显：并行任务数量受模数个数限制。三模数 CRT 最多只有 3 个并行任务，四模数 CRT 最多只有 4 个并行任务，无法填满 8 个核心。此外，若不同模数下的 NTT 运行时间略有差异，较快完成的线程需要等待最慢的模数任务完成后才能进入 CRT 合并。不过由于本实验选择的小模数规模接近、NTT 长度相同，这种负载不均衡通常不严重。因此模数并行更适合作为结构简单、同步开销较小的对照版本。

6.4 CRT 合并中的 guided 调度

CRT 合并阶段使用：

CRT 合并 guided 调度

```
1 #pragma omp parallel for schedule(guided, 256)
2 for (int i = 0; i < result_len; ++i) {
3     u64 r[CRT_CNT];
4     for (int j = 0; j < CRT_CNT; ++j) r[j] = residues[j][i];
5     ab[i] = crt_combine_one(r, target_mod, inv_prod_mod);
6 }
```

guided 调度会在开始时分配较大的任务块，随后逐渐减小任务块大小。相比 static，它更适合负载不完全均匀或执行时间难以完全预测的循环，但调度开销略高。

CRT 合并阶段的循环以输出系数为单位展开，每个 i 都需要从四个小模数结果中取出余数，然后通过 Garner 形式逐步恢复目标模数下的结果。理论上，每个系数的合并步骤数相同，使用 static 也可以得到较好的负载划分。但实际运行时，取模、128 位整数运算和缓存访问可能导致不同迭代的耗时并不完全一致，因此本实验在合并阶段尝试使用 guided 调度。

7 实验五：OpenMP SIMD 自动向量化

7.1 SIMD 优化动机

实验指导中提到，OpenMP 4.0 之后提供了自动向量化方式，可以在内层循环使用 `#pragma omp simd`。在本实验中，点值乘法、逆变换缩放以及部分蝶形循环都具有较强的数据并行性，因此可以尝试自动向量化。

前面几个版本主要利用的是线程级并行，即把不同的模数任务、不同的蝶形块或不同的输出系数分配给多个 CPU 核心执行。SIMD 优化关注的是另一层并行粒度：在单个核心内部，让一条向量指令同时处理多个数据元素。对于 NTT 来说，整体算法虽然存在按层推进的同步关系，但同一层中的大量蝶形运算彼此独立；对于 CRT 版本来说，不同系数的点值乘法和逆变换后的缩放也互不依赖。因此，在已经完成 OpenMP 线程并行之后，继续在内层循环加入 SIMD 提示，可以进一步挖掘每个核心内部的计算能力。

本实验选择在四模数 CRT 的 OpenMP 模数并行版本基础上加入 SIMD，而不是重新改变整体并行结构，主要有两个原因。第一，模数并行能够保持四个小模数 NTT 任务相对独立，代码结构清晰，不会引入复杂的嵌套并行；第二，SIMD 优化只需要作用在每个小模数 NTT 内部的局部循环上，属于

较低层次的优化，不会改变 CRT 合并的数学含义和最终结果。这样可以较好地地区分线程调度优化和向量化优化的作用。

从循环特征看，比较适合加入 `#pragma omp simd` 的位置包括三类：一是点值乘法 $A[i] = A[i] * B[i] \bmod p$ ，每个下标独立；二是逆 NTT 后整体乘以 `inv_limit`，每个元素只读写自身；三是同一个蝶形块内部的 k 循环，在消除旋转因子递推依赖后，每一次迭代访问的两个数组位置互不重叠。需要注意的是，`#pragma omp simd` 只是告诉编译器该循环适合向量化，最终是否真正生成 SIMD 指令仍取决于编译器、优化等级、目标平台以及循环内部运算是否足够规整。

7.2 消除旋转因子递推依赖

原始蝶形循环中存在：

$$w \leftarrow w \cdot wn$$

这会形成循环迭代之间的依赖，不利于编译器向量化。因此在 SIMD 版本中先预计算 `roots` 数组：

这里的依赖并不是蝶形运算本身的数据冲突，而是旋转因子 w 的计算方式造成的循环携带依赖。第 k 次迭代使用的 w 依赖第 $k-1$ 次迭代更新后的结果，因此编译器不能简单地把多个 k 迭代合并成一组向量指令执行。即使 `f[base+k]` 和 `f[base+k+half]` 在不同迭代中没有重叠， w 的递推也会使循环看起来像串行链条。

为了解决这一点，我将每一层所需的旋转因子提前存入 `roots` 数组，使第 k 个蝶形直接读取 `roots[k]`。这样做之后，蝶形循环内部不再更新共享的 w ，每次迭代只依赖自己的数组下标和预先计算好的旋转因子。也就是说，循环从“当前迭代要等待上一次迭代产生 w ”变成了“所有迭代都从只读数组中取自己的旋转因子”，更符合 SIMD 向量化对独立迭代的要求。

预计算旋转因子

```
1 std::vector<u64> roots(half);
2 roots[0] = 1;
3 for (int k = 1; k < half; ++k) {
4     roots[k] = mul_mod_ntt(roots[k - 1], wn, mod);
5 }
```

随后在蝶形循环中直接读取 `roots[k]`：

OpenMP SIMD 蝶形循环

```
1 #pragma omp simd
2 for (int k = 0; k < half; ++k) {
3     u64 x = f[base + k];
4     u64 y = mul_mod_ntt(f[base + k + half], roots[k], mod);
5     f[base + k] = add_mod(x, y, mod);
6     f[base + k + half] = sub_mod(x, y, mod);
7 }
```

预计算旋转因子会额外带来一个长度为 `half` 的数组和一次预处理循环，因此它并不是完全没有代价。对于很小的 NTT 层，`half` 较小，预计算和 SIMD 本身的收益可能被额外开销抵消；但对于大规模样例，例如 $n = 131072$ ，中后期层的蝶形数量较多，循环体重复执行次数大，消除递推依赖后更容易让编译器做循环展开和向量化，收益更有可能体现出来。

此外,蝶形循环写入的是 `base+k` 和 `base+k+half` 两个位置。固定一个 `base` 时,随着 `k` 递增,这两段访问区域分别连续且互不重叠;不同 `base` 对应不同蝶形块,也不会写入同一段数组。因此在同一层内部,消除 `w` 递推依赖后,蝶形主体满足 SIMD 所需的独立性。层与层之间仍然必须保持顺序,因为下一层的输入依赖上一层完整输出,这一点没有被 SIMD 改变。

7.3 小模数乘法优化

CRT 中小模数均小于 1.3×10^9 , NTT 内部参与乘法的数均小于对应模数,因此乘积小于:

$$1.3 \times 10^9 \times 1.3 \times 10^9 \approx 1.69 \times 10^{18}$$

仍在 `uint64_t` 范围内。因此在小模数 NTT 内部可以使用:

小模数 NTT 乘法

```
1 static inline u64 mul_mod_ntt(u64 x, u64 y, u64 mod) {
2     return x * y % mod;
3 }
```

而在涉及目标大模数或通用模乘时,仍然使用:

通用安全模乘

```
1 static inline u64 mul_mod(u64 x, u64 y, u64 mod) {
2     return (u64)((u128)x * y % mod);
3 }
```

8 性能测试与正确性分析

8.1 性能测试与结果分析

为了比较不同并行实现的实际运行效果,本实验在服务器端使用 `perf` 对六个版本进行了性能分析。测试均选取第 4 组大规模样例,即

$$n = 131072, \quad P = 1337006139375617$$

该样例既能体现大规模 NTT 的计算压力,也能检验四模数 CRT 对大模数结果的覆盖能力。编译时统一使用 `-O3 -g -fno-omit-frame-pointer`, Pthread 版本使用 `-pthread`, OpenMP 版本使用 `-fopenmp`。性能统计命令如下:

```
1 perf stat -r 5 ./main 4 4
2 perf record -g -o perf_XXX.data ./main 4 4
3 perf report --stdio -i perf_XXX.data
```

其中,程序自身输出的 `latency` 统计的是多项式乘法核心计算区域;`perf stat` 中的 `time elapsed` 则包含程序启动、读写文件、结果检查等额外开销。因此分析时主要使用程序输出的 `latency` 比较算法主体性能,同时参考 `perf` 的 CPU 利用率、指令数和热点函数判断瓶颈来源。

表 4: 串行与并行版本在大规模样例上的 perf 统计结果

版本	核心计算平均延迟 / ms	perf elapsed / s	CPU 利用率	cycles	instructions	IPC
基础 NTT Pthread	106.69	0.23271	1.948	1.011×10^9	1.585×10^9	1.57
四模数 CRT 串行	927.15	1.0473	0.998	2.560×10^9	4.151×10^9	1.62
四模数 CRT Pthread	318.32	0.4440	2.370	2.575×10^9	4.150×10^9	1.61
OpenMP 模数并行	251.48	0.37019	2.812	2.567×10^9	4.192×10^9	1.63
OpenMP 内部并行	353.28	0.4791	2.469	2.860×10^9	4.987×10^9	1.74
OpenMP + SIMD	144.90	0.27590	2.102	1.360×10^9	1.569×10^9	1.15

从表 4 可以看出, 基础 NTT Pthread 版本的核心计算延迟最低, 但它只进行单模数 NTT, 不需要四模数 CRT, 因此计算量与后面四个 CRT 版本并不完全相同。它更多用于说明基础 NTT 的并行蝶形计算效率。在 perf report 中, 该版本的热点主要集中在 `run_ntt_worker`、`ntt_pthread` 和 `__umodti3` 上, 其中 `pthread_barrier_wait` 占比较低, 说明本次测试中线程同步不是主要瓶颈, 主要开销仍来自 NTT 蝶形运算和模乘。

四模数 CRT 串行版本是后续 CRT 并行版本的对照基线。该版本平均核心计算延迟约为 927.15 ms, perf stat 中 CPU 利用率约为 0.998, 基本等价于单核顺序执行。perf report 显示 `poly_multiply_crt_serial` 的 Children 占比约为 89.40%, 其下主要热点集中在 `multiply_mod_ntt`、`ntt_serial` 和 `__umodti3`。这说明串行 CRT 的主要耗时来自四个小模数下的 NTT 变换和模乘, 而读入、写出、检查结果等外围函数占比较低。因此, 对 CRT 版本做并行优化时, 最有效的方向就是并行化四个模数上的 NTT 计算, 或者优化 NTT 内层蝶形运算与模乘。

四模数 CRT Pthread 版本需要对四个小模数分别执行 NTT, 并在最后进行 CRT 合并, 因此总计算量明显增加。该版本核心计算平均延迟约为 318.32 ms, perf report 显示 `multiply_mod_ntt` 的 Children 占比约为 89.69%, 其中 `ntt_serial` 和 `__umodti3` 占比较高。这说明 Pthread CRT 版本的主要瓶颈不在 CRT 合并本身, 而在每个小模数下的 NTT 计算和 128 位取模乘法。CRT 合并相关函数 `combine_worker`、`run_combine_worker` 占比较低, 说明按输出系数并行合并的开销相对较小。

OpenMP 模数并行版本的平均延迟约为 251.48 ms, 比四模数 CRT Pthread 版本更快。该版本将四个小模数任务直接交给 OpenMP 的 `parallel for` 处理, 每个模数内部保持串行 NTT, 减少了手动创建线程和合并阶段的管理开销。perf report 中 `poly_multiply_crt_openmp_modparallel` 和 `multiply_mod_ntt` 仍是主要热点, 说明计算主体仍然是小模数 NTT; 同时 CPU 利用率达到 2.812, 高于 CRT Pthread 版本, 说明模数层并行确实提高了核心利用率。不过由于四模数最多只有 4 个并行任务, 无法完全填满 8 核, 因此加速幅度仍然有限。

OpenMP 模数串行、NTT 内部并行版本的平均延迟约为 353.28 ms, 是几个 CRT 优化版本中较慢的一个。理论上, 内部并行可以利用更多核心, 但 perf report 中可以看到较多样本落在 `libgomp.so`、`GOMP_parallel`、`ntt_openmp` 和 `__umodti3` 相关路径上。这说明该版本在每一层 NTT 中频繁进入 OpenMP 并行区域, 虽然提高了并行粒度, 但也引入了较多运行时调度和隐式同步开销。对于本实验规模和实现方式而言, 这部分开销抵消了内部并行带来的收益, 因此实际性能不如模数并行版本。

OpenMP + SIMD 版本在 CRT 版本中表现最好, 核心计算平均延迟约为 144.90 ms, perf elapsed 也降至 0.27590 s。与 OpenMP 模数并行版本相比, 该版本的 cycles 从约 2.567×10^9 降到约 1.360×10^9 , instructions 从约 4.192×10^9 降到约 1.569×10^9 。perf report 中主要热点集中在 `ntt_serial`, 而 `__umodti3` 占比明显下降, 这与代码中使用小模数乘法 `mul_mod_ntt`、预计算旋转因子并加入 `#pragma omp simd` 的优化方向一致。虽然 `#pragma omp simd` 不保证一定生成最优向量指令, 但本次测试结果说明, 减少 128 位模乘并改善内层循环结构后, 整体执行指令数和周期数都有明显下降。

综合来看, 在本次大规模样例上, 各版本按核心计算延迟从低到高大致为: 基础 NTT Pthread、

OpenMP + SIMD、OpenMP 模数并行、四模数 CRT Pthread、OpenMP 内部并行、四模数 CRT 串行。基础 NTT Pthread 由于不是 CRT 大模数版本，只能作为单模数 NTT 并行效果参考；若只比较支持大模数的四模数 CRT 版本，则 OpenMP + SIMD 最优，模数并行次之。

串行与并行版本的对比更加直接地体现了并行优化的效果。以四模数 CRT 串行版本的 927.15 ms 为基准，四模数 CRT Pthread 版本缩短到 318.32 ms，加速比约为 2.91；OpenMP 模数并行版本缩短到 251.48 ms，加速比约为 3.69；OpenMP 内部并行版本缩短到 353.28 ms，加速比约为 2.63；OpenMP + SIMD 版本进一步缩短到 144.90 ms，加速比约为 6.40。这些结果说明，CRT 问题中四个小模数 NTT 天然相互独立，采用模数层并行可以用较少同步开销获得稳定收益；而只在 NTT 内部每一层蝶形循环上展开并行时，虽然并行粒度更细，但频繁的并行区域创建和层间隐式同步会降低收益。进一步结合 SIMD 和小模数乘法优化后，既减少了每次蝶形运算的指令开销，又保留了模数层并行的粗粒度特征，因此整体速度提升最明显。

8.2 Pthread 和 OpenMP 区别分析

Pthread 和 OpenMP 都可以用于多线程并行优化，但二者的编程方式和性能特点不同。Pthread 属于较底层的线程库，需要程序员手动创建线程、划分任务、设置同步机制并回收线程，因此代码实现较复杂，但线程控制更加精细，适合对同步位置和任务粒度有明确要求的场景。OpenMP 则属于较高层的并行编程接口，只需要在循环前添加 `#pragma omp parallel for` 等制导语句，即可由编译器和运行时系统自动完成线程创建、任务分配和同步，代码更加简洁，适合循环结构规则、迭代之间相互独立的计算任务。

从本实验结果看，OpenMP 并不一定总是优于 Pthread，关键取决于并行粒度和同步开销。对于 CRT 模数并行，不同小模数下的 NTT 计算相互独立，任务粒度较粗，OpenMP 可以通过 `parallel for` 直接将不同模数任务分配给线程，调度开销较小，因此性能优于四模数 CRT 的 Pthread 版本。而对于 NTT 内部并行，每一层蝶形变换结束后都必须同步，OpenMP 若频繁进入并行区域并进行隐式同步，会产生较明显的运行时调度开销，因此该版本性能反而不如模数并行版本。

Pthread 的优势在于可控性强，例如可以手动设计固定线程数、任务划分方式和 barrier 同步位置，减少不必要的线程管理开销；缺点是代码量较大，调试难度更高。OpenMP 的优势在于实现简单、可维护性好，并且可以方便地尝试 `static`、`dynamic`、`guided` 等不同调度策略；缺点是运行时调度机制对程序员不完全透明，在细粒度循环中可能带来额外开销。综合来看，Pthread 更适合需要精细控制线程行为的并行算法，OpenMP 更适合规则循环和粗粒度任务并行。在本实验的 NTT/CRT 场景下，OpenMP 模数并行和 OpenMP+SIMD 优化效果较好，而频繁同步的内部 NTT 并行需要谨慎使用。

9 其他算法优化策略

除本实验已经实现的 Pthread、OpenMP 模数并行、OpenMP 内部并行和 SIMD 自动向量化之外，NTT 与 CRT 多项式乘法还可以从以下几个并行角度继续优化。

9.1 任务池与动态负载均衡

当不同任务的计算量不完全一致时，可以使用任务池进行动态调度。例如在 NTT 后几层中，block 数量较少，如果仍然按 block 静态划分，可能出现部分线程空闲。可以将每个 butterfly block 或一段连续 butterfly 操作作为任务放入队列，线程完成当前任务后继续取下一个任务执行。这样可以减少尾

部等待，提高负载均衡效果。不过任务池需要使用锁、原子变量或条件变量维护任务队列，因此适合任务粒度较大、负载差异较明显的情况。即任务池操作。

9.2 分层混合并行

CRT 版本天然包含两层并行：外层是不同小模数之间的模数并行，内层是单个小模数 NTT 的蝶形并行。本实验为了避免线程数超过 8，没有使用嵌套并行。进一步优化时，可以采用分层混合策略，例如四个模数各分配 2 个线程，或在模数数量较少时把剩余线程用于当前模数内部 NTT。这样既能利用模数之间的独立性，也能在单个模数任务较重时提高核心利用率。该方法的关键是控制总线程数，避免 OpenMP 嵌套并行导致线程过量创建。

9.3 缓存友好的分块优化

NTT 的访存模式会随着层数变化而变化。前几层访问距离较短，缓存命中率较高；后几层跨度增大，可能出现更多缓存缺失。可以尝试将连续的 butterfly 操作按缓存大小分块，使每个线程尽量处理连续内存区域，减少不同线程访问相邻缓存行造成的伪共享。对于 CRT 合并阶段，也可以让每个线程处理连续的输出系数区间，从而提高顺序访存比例。

9.4 多次实验任务并行

实验测试通常包含多组输入样例。如果每组样例之间互不依赖，也可以在更高层进行任务并行，即不同线程或不同进程分别处理不同测试样例。这种方式不会改变单次 NTT 的内部结构，实现简单，适合批量测试或离线评测。但在本实验平台中，每个提交任务只申请 1 个 CPU，因此这种方法需要谨慎控制并发数量，避免多个样例同时运行时相互抢占核心资源。

总体而言，NTT 并行优化需要在并行粒度和同步开销之间取得平衡。粒度太粗时，核心利用率不足；粒度太细时，线程调度、同步和缓存开销会抵消计算收益。因此更理想的策略通常是：外层使用较稳定的模数并行或任务并行，内层只在计算量足够大时进行蝶形并行，并结合 SIMD 与缓存友好访问进一步降低常数开销。

10 问题与反思

10.1 关于三模数与四模数

三模数合并符合讲义说明，但对于大模数并不一定足够。CRT 的关键不是目标模数是否大于小模数，而是小模数乘积是否大于真实卷积系数上界。对于 $P = 1337006139375617$ 的测试，三模数乘积明显不够，因此必须使用四模数。

10.2 关于模数并行与内部并行

CRT 的模数层并行非常自然，但四模数最多只有 4 个并行任务，而平台有 8 核。对于 $n = 131072$ ，单个 NTT 内部每层有大量蝶形操作，因此内部并行通常更适合充分利用核心。模数并行适合作为对照版本，优点是结构简单、无嵌套并行。

11 总结

本实验从朴素卷积出发，逐步实现了 NTT、CRT、Pthread、OpenMP 和 SIMD 自动向量化多个版本。基础 NTT 将复杂度降为 $O(n \log n)$ ，并初步进行了并行计算的测试；CRT 版本解决了大模数或非 NTT 友好模数下的计算问题；Pthread 版本采用主线程参与计算的方式，符合 8 核平台约束；OpenMP 版本进一步利用 `static`、`guided` 和 `simd` 实现自动调度与向量化尝试。

通过调试可以看出，CRT 模数选择对大模数正确性非常关键。四模数版本中，将第四模数替换为 1224736769 后，模数乘积显著增大，最终通过大模数样例。整体而言，本实验不仅完成了 NTT 多项式乘法的并行优化，也加深了对线程粒度、负载均衡、同步开销、CRT 覆盖范围和编译器自动向量化的理解。